

5. Image Classification with CNNs (1)

흐름: 수작업 Image Feature → ConvNet의 등장 → Convolution Layer → Spatial Dimension과 Receptive Field → Downsampling → Pooling → Translation Equivariance

1. Image Classification & Feature Representation

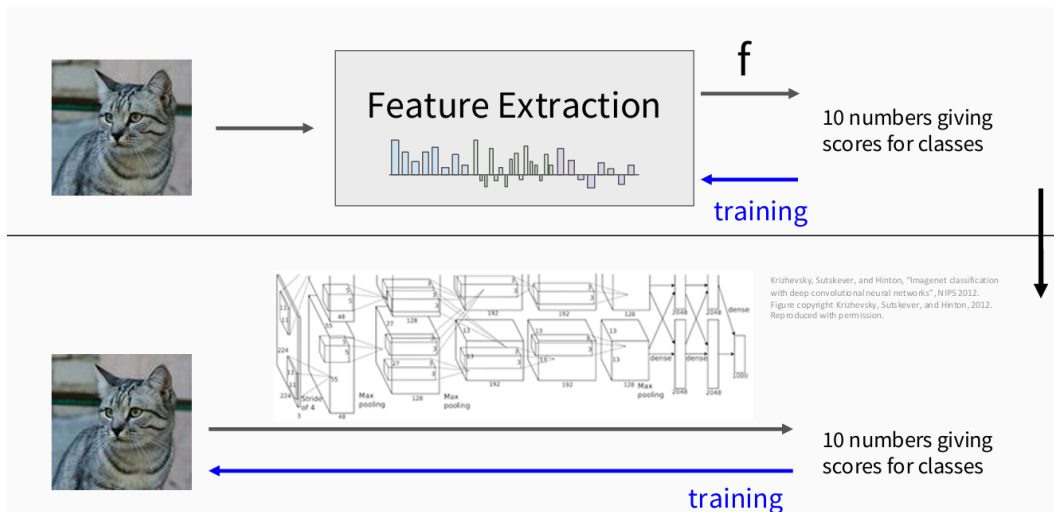
Image Classification: 주어진 label 집합에서 이미지 하나의 class를 예측
feature extractor 설계 → 추출된 feature x로 linear classifier 학습

Image Features

- **Color Histogram:** 이미지의 색 분포를 histogram으로 표현.
pixel의 spatial structure X. 비슷한 색을 가진 서로 다른 물체를 구분하기 어려움
- **Histogram of Oriented Gradients(HoG):** 이미지를 작은 영역으로 나눠, 각 영역의 edge 방향을 histogram으로 표현.
color 정보 X. edge, local structure 표현 적합.
- **Bag of Visual Words:** 여러 이미지 → local patch 추출 → visual word codebook 생성. 각 이미지에 어떤 visual word가 얼마나 나타나는지 histogram으로 표현.

ConvNets (CNN)

- 기존 방식: image → 사람이 설계한 feature extraction → classifier 학습.
- ConvNet: feature extractor + classifier 둘 다 end-to-end로 학습.

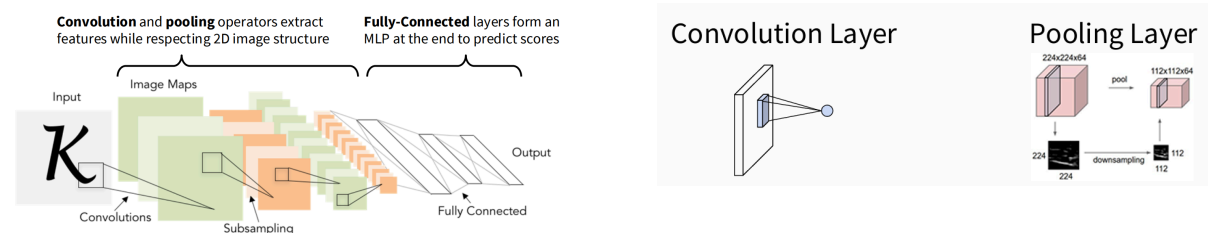


2. Convolutional Neural Network(CNN)

MLP: 1차원 vector 표현 → spatial 정보 삭제

CNN: **convolution + pooling** → 2D 공간 구조를 유지 + feature 추출 가능

- Convolution/Pooling: 이미지의 공간 구조를 유지하며 feature 추출
- Fully Connected Layer: 마지막 feature를 이용해 class score 예측
- 전체 모델은 backpropagation과 gradient descent로 end-to-end 학습



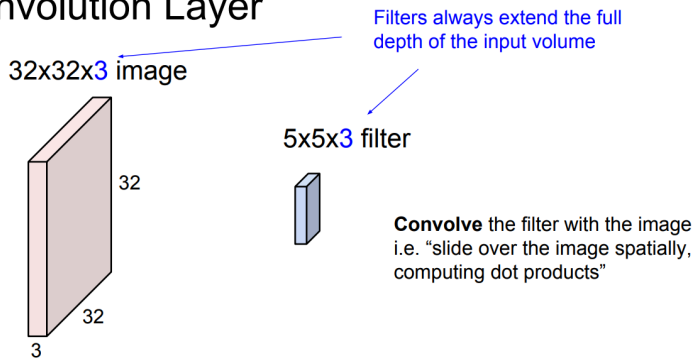
역사: LeNet → AlexNet → ConvNet → ViT (Vision Transformer)

3. Convolution Layer

3.1 연산 방식

- filter 크기(H, W) → 자유.
- depth(channel) → image channel과 동일하게 설정.

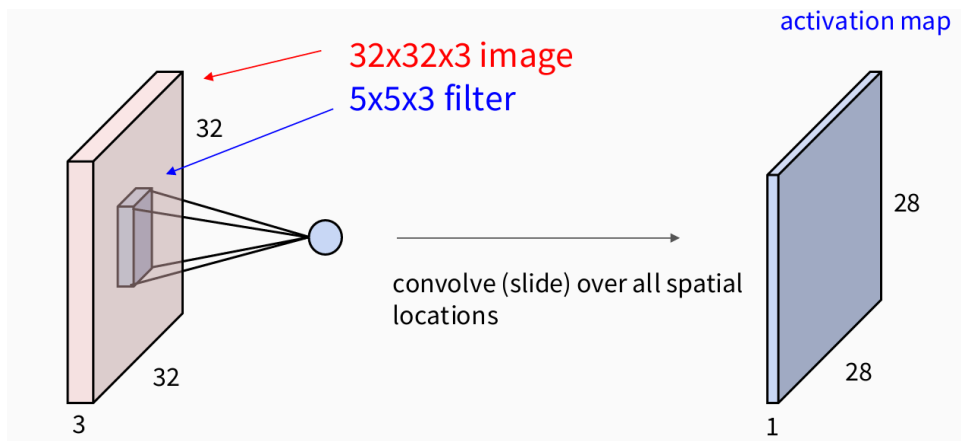
Convolution Layer



[기본 연산]

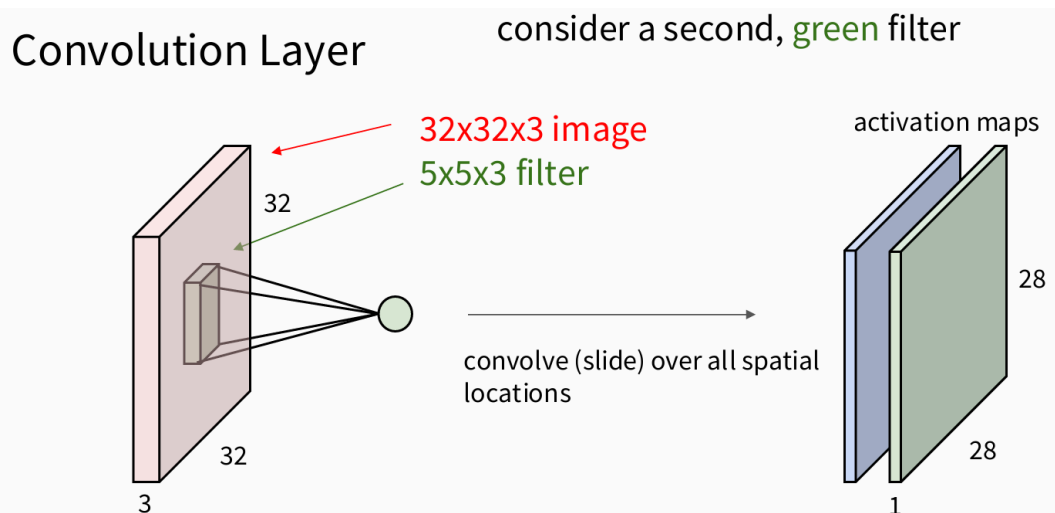
filter 이동 → 각 local image patch와 filter의 dot product 계산 + bias → 하나의 scalar 출력

⇒ 하나의 **activation map** 생성



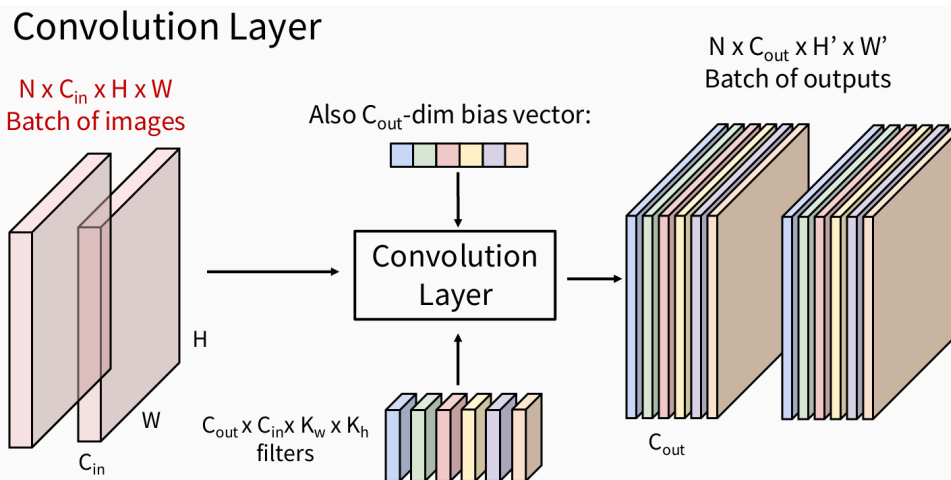
여러 개의 Filter

각 filter가 하나씩 activation map 생성.



bias, Batch까지 포함한 일반 형태

- bias: filter 개수만큼 존재
- batch: 한 번에 처리하는 image 개수



ConvNet

- (각 layer 연산 + activation function 적용) * N layer 쌓기
- Activation function $X \Rightarrow$ linear operation

3.2 Filter의 학습 목표

- Linear Classifier: class마다 전체 이미지를 대표하는 template 하나 학습.
- MLP: 여러 종류의 whole-image template. (local spatial structure 활용 X.)
- Convolutional Filter
 - 첫 번째 Convolution Layer: 작은 local image template 학습.
 - 깊은 Convolution Layer: 거시적 관점(더 큰 구조) 학습.

4. Spatial Dimension: Padding, Stride

4.1 Padding

Padding: feature map의 shrink(크기 축소) 방지.

- zero padding: 입력 주변에 p개의 0 채우기.

- same padding: 입력 image와 출력 image의 size를 맞추는 방식. $P = \frac{K-1}{2}$ 개 필요 (stride=1 기준).

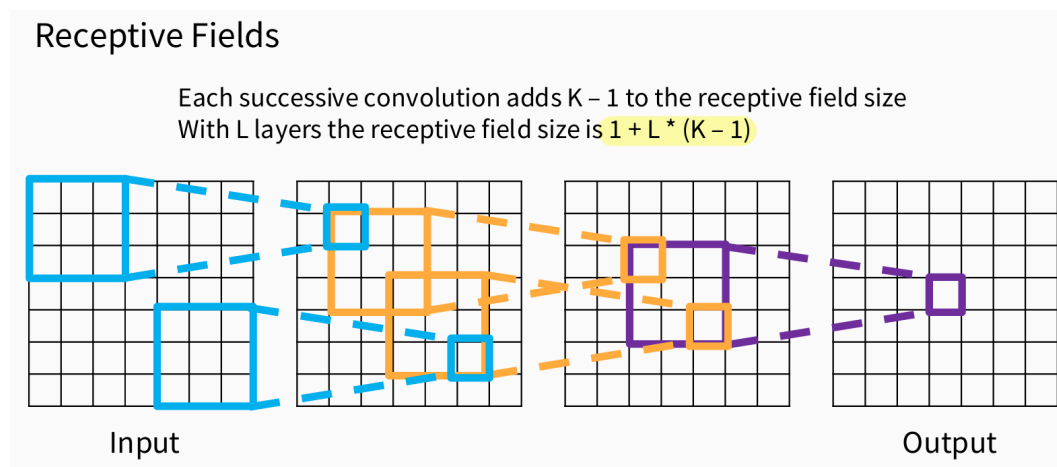
Output 크기 연산: $O = \frac{W-K+2P}{S} + 1$

(W: 입력 크기. K: kernel 크기. P: padding 크기. S: Stride.)

4.2 Receptive Field

receptive field: 출력 원소 하나가 입력의 어느 영역에 의존하는지 나타내는 영역

- kernel size K → (convolution 출력 1개에 대해) 이전 layer의 $K \times K$ 영역 참조.
- L개 layer의 receptive field의 size: $1 + L(K - 1)$



문제: 큰 이미지에서 출력이 전체 이미지를 보게 하려면 많은 convolution layer 필요.

⇒ network 내부의 downsampling. (**Stride, Pooling**)

4.3 Stride

Stride: filter의 이동 간격.

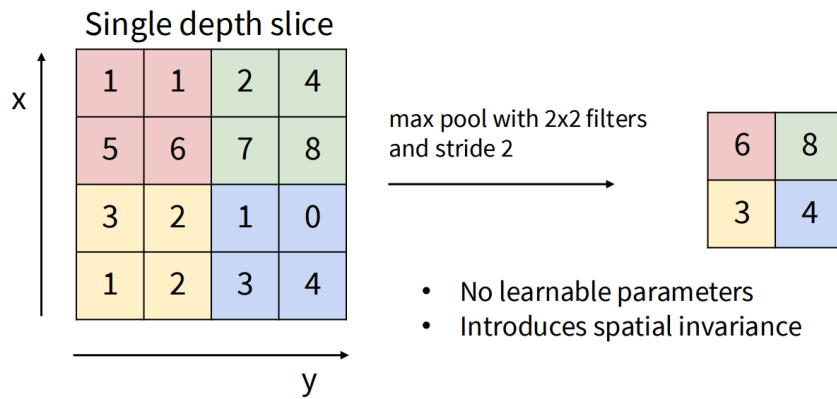
4.4 Pooling Layer

Pooling: activation map의 spatial dimension을 downsample하는 방법.

- 각 channel을 독립적으로 처리 (channel 수 C 유지)
- learnable parameter 없음

Max Pooling: 설정한 영역 중, 가장 큰 값 선택. nonlinearity 연산.

→ 작은 spatial shift에 대해 representation이 덜 민감.



Pooling 출력 크기: $I' = \frac{I-K}{S} + 1$
 (입력 image 크기 I. kernel 크기 K. Stride 크기 S)

5. Translation Equivariance

Convolution이나 pooling을 적용하기 전에 입력을 이동한 결과 = 연산 후 출력을 이동한 결과

Intuition: image의 feature ← location에 의존하지 않음/

Convolution and Pooling: Translation Equivariance

